ROUTE YAPILANMASI

MVC'DE ROUTE YAPISI

ASP.NET DEFAULT ROUTE(MAPDEFAULTCONTROLLERROUTE) YAPISI

MAPCONTROLLERROUTE ILE CUSTOM ROUTE

STATİK DEĞER İLE RORA BELİRLEMEK

CUSTOM ROUTE TASARLAMA YAKLAŞIMI

PARAMETRELER ILE VERI ALMA

Route Parametrelerinden Gelen Değerleri Action'da Karşılama

ROUTE CONSTRAİNTS

Route Constraints ile Tür Kısıtlaması

Route Constraints ile Yapısal Kısıtlamalar

Custom Constrait

Custom Constrait Oluşturma

Custom Constrait Kullanımı

Custom Constrait Çalışma Mekenizması

ATTRİBUTE ROUTİNG

ATTRİBUTE ROUTİNG OLUŞTURMA

CUSTOM ATTRİBUTE ROUTİNG OLUŞTURMA(CONTROLLER)

Action Bazlı Attribute Routing Oluşturma

ROUTİNG HAKKINDA BİLGİLER

CUSTOM ROUT HANDLER

CUSTOM ROUT HANDLER OLUŞTRUMA

CUSTOM ROUT HANDLER SINIFI OLUŞTRUMA

CUSTOM ROUT HANDLER ILE RESIM BOYUTLANDIRMA ÖRNEĞI

Route Yapılanması

Route, gelen isteğin rotasını belirleyen yapılanma.

Gelen istek, nereye? Ne şekilde? Hangi formda? Geleceğini belirleyen şablonlardır.

MVC’de Route Yapısı

Startup.cs dosyası içerisindeki “app.UseRouting()” fonksiyonu bu işlemi gerçekleştirmektedir.

UseRouting, tetiklendiği zaman gelen Request’teki rotaları ayırır ve ilgili rotaya karşılık gelen Controller’ı ayağa kaldırır sonrasında ilgili Action’ı tetikler. Tetiklenme mekanizması ve işlemlerin uygulanması/Asp.Net Core Pipeline, MiddleWare kısmında anlatılacak.

UserRouting’in isteği karşılayıp ilgili yönlendirmeyi yapabilmesi için önceden bir rota tanımlanmış olması gerek.

Asp.Net Core MVC içerisinde, UseEndpoint fonksiyonu(Eskiden UseEndpoints’di şimdi kullanılmıyor.) ile tanımlanan default olarak gelen bir rota vardır.

Yani özetle gelen isteğin ayrıştırma işlemi UseRouting() fonksiyonu ile yapılırken, Rotalar UseEndpoints()(Yeni versiyonda direkt olarak app.MapControllerRoute/MapDefaultControllerRoute fonksiyonu kullanılıyor UseEndpoints’e gerek yok.) ile tanımlanıyor.

Bir MVC projesinde default olarak gelen bir Route yapısı vardır. Mimarinin içinde gelen default olarak tanımlanmış Route’u kullanmak için

MapDefaultControllerRoute fonksiyonu kullanılır. Zaten Proje açıldığı zaman program.cs içerisinde bu fonksyion kullanılır.

Asp.Net Default Route(MapDefaultControllerRoute) Yapısı

MVC mimarisinde gelen default route tanımalamsıdır. Parametre almaz sadece rotanın “Controller=Home”/”Action=Index” olduğunu bildirir.

Bu yapı default Asp.Net Core mimarisinin içinde gelen default rotadır.

Rota oluşturulurken süslü parantez içerisindeki değerler parametre olarak algılanır mimari tarafından.

Parametreler içerisinde ismi “controller” veya “action” olan varsa bunlar mimarinin ön tanımlı parametrelerdir.

Bu parametreler dışındaki parametreler Custom parametre olarak geçer.

Yani sistem adı “controller” ve “action” olan parametreleri Controller ve Action olarak algılar bir nevi tanır. Kalan parametreleri ise custom olarak sayar.

Bu controller ve action parametrelerinin default değerleri de atanabilir. Yani controller için bir spesifik default controller ve action için spesifik bir default action. Bu sayede Request’de hiçbir Action veya Controller verilmese bile yani rota boş geldiği zaman, gidilecek olan default konumu bildirir. Örn: {controller=Home}/{action=Index}

Bu sayede controller Request’de controller gelmediği durumda Home Controller ayağa kaldırılır ve içerisindeki Index Action’ı tetiklenir.

Bunun yanı sıra Request içerisinde Controller’ın belirtildiği fakat Action’ın belirtilmediği durumda, ilgili Controller içerisinde “Index” Action’ı aranır ve tetiklenir. Çünkü default değer olarak Index verilmiş Route’da.

Gönderilen	Açılan
/	/Home/Index
/Home	/Home/Index
/Home/Index	/Home/Index
/Product	/Product/Index -> home ezildi

/Product/Getir	/Product/Getir -> ikisi de ezildi

MapControllerRoute ile Custom Route

Custom Route oluşturmak için MapControllerRoute() (Eskiden "endpoints.MapControllerRoute()", Şimdi "app.MapControllerRoute()") fonksiyonu kullanılır.

MapControllerRoute fonksiyonu default'ın dışında route oluşturan bir fonksiyondur.

MapControllerRoute parametre olarak isim ve patern bekler.

Her bir ControllerRoute'un bir ismi olması gerekiyor. Bu sebeple MapControllerRoute, parametre olarak bir isim bekler. MapDefaultControllerRoute rotasının da bir ismi vardır, ismi çok beklenemedik bir şekilde "default"tur.

Pattern kısmı ise Custom olan kısımdır yani düzenin belirlendiği.

Örenğin:

```
app.MapControllerRoute("CustomRoute", "{action}/{controller}");
```

Şu anda bu tanımala sayesinde önce action beklenir sonra controller. Yani Index/Home beklenir mesela.

Buradaki tüm kurallar isteğe göre ayarlanabilir.

Örneğin:

```
app.MapControllerRoute("CustomRouteMAL", "{action}/SaDunya/{controller}");
```

Şu anda Index/SaDunya/Home şeklinde çalışır. Yani arada SaDunya değeri olamayan hiçbir Request kaşılanmaz. O yüzden böyle mal mal şeyler yapılmaz adam akıllı tanımlanır.

MapDefaultControllerRoute'da olduğu gibi Custom route tanımlarken de default Controller'lar ve Action'lar verilebilir.

Örneğin:

```
app.MapControllerRoute("CustomRoute", "{controller=Product}/{action=Listele}");
```

Burada default Controller Product oldu yani Request'de Controller bilgisi gelmezse Product Controller ayağa kaldırırlır.

Eğer "controller" ve "action" parametrelerine defualt değer verilmezse, boş Request geldiği takdirde mimari hata verecektir. Bu sebeple tanımlamak gerekir.

Route içerisinde statik değer tanımalamak direkt hata çünkü boş geldiği zaman Request, mimari default olarak tanımlanan controller ve action'ı açamaya çalışacak fakat arada bir statik değer varsa patlar çünkü boş Request adı gereği boş olduğu için içinde o konumda o statik değeri barındırmaz.

```
("CustomRoute", "{controller=Product}/SalakcaBirStatikDeger/{action=Listele}");
```

Bu route sadece Controller ile Action arasında "SalakcaBirStatikDeger" verisi olan Url'lerde çalışabilir. Bu sebeple böyle işler yapmıyoruz.

Statik Değer ile Rora Belirlemek

Url'de hiçbir parametre kullanmadan direkt olarak statik değer ile(anahtar kelime gibi düşünülebilir) custom rota yapmak için kullanılır.

Burada MapControllerRoute 3 adet paramtere alır. İlki klasik olarak Route ismi. İkincisi statik olan paramtere(anahtar kelime). Üçüncüsü anonim olarak verilen controller ve action'lar.

Örneğin:

```
app.MapControllerRoute("AnasayfaRoute", "Anasayfa", new { controller =
"Home", action = "Index" });
```

bilmemNe.com/anasayfa yazınca HomeController içideki Index'e gider.

Burada statik olan bir Url'e arkada yönlendime sağlanır yani Url'den parametre alınmaz. Günlük hayatta kullanıldığı söylenilebilir.

Bu Route yapılanması sadece gelen Url ile ilgili değildir çift taraflıdır. Örneğin Proje içerisinde TegHelper ile Home Controller'ına ve Index Action'ına yönlendirilirse kullanucı link'e bastığında Url'de "site/Anasayfa " görür.

Custom Route Tasarlama Yaklaşımı

Birden fazla rota tanımlanacaksa, bu rotalar özelden genele tanımlanır.

Yani en özel, en spesifik rota en üstte tanımlanmalı. En genel (bkz. MapDefaultControllerRoute) en altta tanımlanmalı. Çünkü önce özeller kontrol edilmeli ki genel kontrollü en son durum yapılsın if-else mantığı.

Örneğin:

```
app.MapControllerRoute("AnasayfaRoute", "Anasayfa", new { controller = "Home",
action = "Index" });
app.MapControllerRoute("ListeleRoute", "{controller=Product}/{action=Listele}");
app.MapControllerRoute("ActionFirstRoute", "{action}/{controller}");
app.MapControllerRoute("CustomRouteMAL", "{action}/SaDunya/{controller}");
app.MapDefaultControllerRoute();
```

Her bir Route'un ismi uiniq/özel olmalıdır.

Parametreler ile Veri Alma

Süslü parantez ile girilen paramereler aslında mimari için bir ayrı bir anlam taşır. Bunu bir değişkene benzetebiliriz.

Mesela {controller} ve {action} kısımları için konuşursak. Url'den bir controller ve bir action geldiği zaman sırası ile bu değerlerin mimari için bir anlamı oluyor. Ayrıca buradaki Controller ve Action değerleri değişebiliyor yani static değil, bir kere Home gelip sonrasında Product gelebiliyor.

Aynı mantık ile controller ve action parametreleri gibi yeni parametreler tanımlanabilir. Mesela Id.

```
app.MapControllerRoute("DataRoute","{controller=Home}/{action=Index}/{x}/{y}/{id?}");
```

Şu anda bu rota için action'dan sonra gelecek olan değerlerin bir anlamı varıdır. Yalnız burada Id parametresi nullable iken x ve y parametreleri nullable değildir. Eğer Url içerisinde x ve y'ye karşılık gelecek değerler gelmez ise mimari hata verir.

Örenğin: “home/index/54/ahmet” Url’i verildiği zaman x değerine 54, y değerine ahmet, id değerine ise hiçbir şey gelmez ve kod hata vermez çünkü id nullable. Bu parameterelerin türü yoktur Action’da belirlenir.

Route Parametrelerinden Gelen Değerleri Action’da Karşılama

Route parametrelerinin türü yoktur çünkü Url üzerinden gelen veriler, gidilen Action’da alınır ve karşılanacak türler orada belirlenir.

Burada iki yol vardır.İlk yol Action’ın parametrelerine ilgili Route’un parametreleri ile aynı isime sahip parametreler verilir.

İkinci yol ise ilgili Route parametreleri ile aynı isime sahip olan property’lere sahip Class ile karşılama yapılabilir.

Genelde ilk yol tercih edilir.

Örneğin:

```
MapControllerRoute("DataRoute", "{controller=Home}/{action=Index}/{x}/{y}/{id?}")
public IActionResult Index(int x, int y, int id)
```

Burada gelecek olan değerlerin hangi türde geleceği bilinmediği için karşılanabilecek en geniş tür ile karşılamak daha doğrudur int yerine string gibi. Çünkü kullanıcı burada kafasına göre “mahmut” yazarsa Url’nin son kısmına parametre olarak. Sistem patlar.

```
public IActionResult Index(string x, string y, string id)
```

Fakat bununda bir level üstü var. Yani gelcek olan parametre değerlerini kısıtlamak. Buna Route Constraints deniyor.

Route Constraints

Url üzerinden gelcek değerleri kısıtlamak için kullanılır. Mesela Id değerinin sadece int kabul edileceği bildirilir. Route’lar için Validation gibi düşünülebilir.

Route tanımlarken parametrelerin içerisine “{parametre: tür} ” ile tanımlanır. “{Id: int}” gibi. Hatta nullable yapmak için “{Id: int?}”.

Route Constraints ile Tür Kısıtlaması

```
("ConstraintRoute", "{controller=Home}/{Action=Index}/{id:int?}/{name?}")
```

Burada şöyle bir mantık kurulması lazım. Mantalite karşılanabilecek en geniş tür olduğu için short yerine int ile karşılama yapılır. Mesela string ile bir Constraint yapılmaz çünkü zaten gelen her değer string ile karşılanabildiği için string ile böyle bir kullanım yoktur. Sadece belirli türler kullanılabilir.

Sayısal değerler için int, zaman değeleri için Datetime, ondalıklılar için Float veya Double, UniqueİdentifyTürler için Guid, anahtar durumları için Bool, sadece ingilizce olan büyük veya küçük harfler için Alpha türleri kullanılır. Bunların dışındaki türler Route Constraint için kullanılamıyor.

Bu Route'da int olması gereken bir Parametreye string değer verilirse kullanıcı 404 hatası alır.

Bu tür kısıtlmalarından sonra Action tarafında parametrelerdeki türleri istenilen türde ayaralanbilir. Çünkü artık başka tür gönderilirse 404 hatası verecek sistem ve Action'a hiç ulaşmayacak bu değerler yani uygulama patlamayacak kullanıcı patlayacak.

Route Constraints ile Yapısal Kısıtlamalar

Tür kısıtlarının dışında aynı Validation'lara benzer şekilde maksimum uzunluk gibi kısıtlamalar da verilebilir Route Constraints ile.

Parametrelerin içerisinde "{Parametre.fonksiyon()}" şeklinde kullanılır. Mesela "{name.lenght(11)}". Bu fonksiyon gelecek olan Parametrenin tam olarak kaç karakterli olması gerektiğini belirtiyor. Mesela TC kimlik alırken tam 11 hane olması gerek ne bir eksik ne bir fazla.

```
("ConstraintRoute", "{controller=Home}/{Action=Index}/{id:int?}/{name:lenght(12)?}")
```

Veya maksimum karakter belirtmek için "{name.maxlenght(11)}". Kullanılabilir. Ayrıca bu Constaint'ler kombinasyon da yapılabilir.

Mesela “{tc:int:lenght(11)}” bu Constraint, tc parametresinin sadece int türde olabileceğini ve uzunluğunun 11 olması gerektiğini bildirir.

constraint	Inline	Class	Notes
length(length)	{id:length(12)}	LengthRouteConstraint	Constrains a route parameter to be a string of a given length or within a given range of lengths.
maxlength(value)	{id:maxlength(8)}	MaxLengthRouteConstraint	Constrains a route parameter to be a string with a maximum length.
minlength(value)	{id:minlength(4)}	MinLengthRouteConstraint	Constrains a route parameter to be a string with a maximum length.
range(min,max)	{id:range(18,120)}	RangeRouteConstraint	Constraints a route parameter to be an integer within a given range of values.
min(value)	{id:min(18)}	MinRouteConstraint	Constrains a route parameter to be a long with a minimum value.
max(value)	{id:max(120)}	MaxRouteConstraint	Constrains a route parameter to be an integer with a maximum value.

Custom Constrait

Hali hazırda gelen Constraint’lerin yeterli gelmemesi durumunda Custom Constraint’ler oluşturulabilir.

IRouteConstraint interface’i ile yapılır. Fakat tabi ki de bu iş için bu interface’den miras alan bir concrete sınıfı oluşturmak gerekiyor.

Bunun için proje altında Constraints klasörü oluşturulur ve içerisine ilgili sınıf oluşturulur. Bu sınıfın ismi Constraint ile bitmesi zorunlu değil. Raconsaldır.

Oluşturulan sınıfa IRoutConstraint interface’i implemente edilir.

İmplemente sonucu bir Match fonksiyonu gelir.

Bu Match fonksiyonu:

- httpContext
- route
- routeKey
- values
- routeDirection

parametrelerini içerir. Ve ilgili Constraint'lerin sağlanması halinde return eder.

Custom Constrait Oluşturma

Custom Constrait sınıfını oluşturduktan sonra bu sınıfı program.cs içerisinde bir servis olacak tanıtmak gerekiyor.

```
builder.Services.Configure<RouteOptions>(options =>
options.ConstraintMap.Add("custom", typeof(CustomConstrains)));
```

Bu kod aslinda Constraint havuzuna custom oluşturulan Constraint'i ekler.

Custom Constrait Kullanımı

Route tanımlarken normal bir Constraint fonksiyonu gibi kullanılır tek farkı başka Constraint'ler ile kullanılamaz sadece Custom Constraint tek başına kullanılabiliyor.

Custom Constrait Çalışma Mekenizması

İstek geldiğinde Controlle'a gitmeden önce ilgili Custom Cosntraint sınıfının içerisine girer. Burada gerekli kontroller sağlanır ve bir sorun olmaması durumunda isetenilen Controller'a yönlendirilir. Sanki ara bir filtre/middleware gibi çalışır.

RouteKey, ilgili Constraint'in hangi parametre için kullanıldığını gösterir.

Values, gelen değerleri gösterir. İçine bakınca tek tek Controller, Action, varsa id ve diğer parametrelere karşılık gelen değerlere erişilebilir.

Tüm kontroller yapıldıktan sonra ilgili Custom Constraint class'ının içerisindeki Match fonksiyonu eğer True dönerse Url geçerli sayılır ve yönlendirmeye devam edilir.

Eğer False dönerse Url geçersiz sayılır 404 hatası verir kullanıcıya.

Attribute Routing

Route'ları program.cs dosyası içerisinde tutmaktansa ayrı bir dosya içerisinde tutmak için kullanılır.

Program.cs içerisinde tanımlama daha geleneksel ve günlük hayatta daha çok kullanılır.

Attribute'ler ile Controller bazlı, Controller'lara özel Route tanımlaması yapılabiliyor.

Yani Controller, kendini tetikleyecek olan rotayı kendi belirler.

Attribute Routing Oluşturma

İlgili Controller'ın üstüne Route Attribute'u oluşturulur.

Attribute Routing oluşturmak için iki seçenek vardır.

Parametreler MapControllerRoute'un aksine "{}" iler ile değil "[]" ile tanımlanır.

Yani "[controller]/[action]" fakat eğer ön tanımlı parametrelerin dışında custom bir özel parametre oluşturulmak istendiğinde "{}" kullanılır.

"[controller]/[action]/{id?}" gibi.

Default bir Route tanımalamsı yapmak için:

```
namespace P07_Route_Yapilanmasi.Constraints
{
    [Route("{controller}/{action}")]
    0 references
    public class AttributeRoutingController : Controller
    {
        0 references
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

Bu tanımlamadan sonra program.cs içerisine bu tanımalamyı bildirmek gerekiyor. `app.MapControllers();` Controller ile Routing yapacağını bildirir.

Custom Attribute Routing Oluşturma(Controller)

Attribute içerisindeki rota parametreleri özelleştirilebilir. Mesela ana/Index denildiğinde ilgili Controller veya Action'a gidilebilir.

Örneğin: `[Route("ana/[action]")]`

Yani Url'e ana yazıldığı zaman ilgili Controller ayağa kaldırılır.

Action Bazlı Attribute Routing Oluşturma

Aynı Controller'larda olduğu gibi Action'larda da özel Route oluşturulabiliyor.

Aynı Controller'larda olduğu gibi Route Attribute'u ile yapılır.

```
public class AttributeRoutingController : Controller
{
    0 references
    public IActionResult Index()
    {
        return View();
    }

    [Route("in")]
    0 references
    public IActionResult AtionBaseRouting()
    {
        return View();
    }
}
```

AttributeRouint/in Url geldiği zaman AtionBaseRouting Action'ını çalışacak.

Keza Geleneksel yöntemde olduğu gibi Action bazlı Routing'de de custom parametre alınabilir.

```
[Route("in/{id:int?}")]
public IActionResult CustomParamater(int? id)
```

Custom parametre verilir ve hemen Action'da da tanımlanır. Ayrıca burada değer nullable gelebildiği için Action'da da değişken nullable belirtilir.

Constraint fonksiyonları falan hepsi klasik yöntem ile aynı şekilde çalışır.

Routing Hakkında Bilgiler

Genellikle MVC'de Routing'ler Program.cs içerisinde tutulur geleneksel olarak.

API yapılanmalarında ise Routing'ler genellikle Controller'ların içerisinde Controller bazlı veya Action bazlı olarak tutulur.

Custom Rout Handler

Custom Route Handler'ın amacı, Controller ile cevap verilmek istenmeyen veya gerek duyulmayan durumlarda özel oluşturulan bir sınıf ile isteğin karşılanmasıdır.

Custom Rout Handler Oluştruma

Program.cs içerisinde app.map() fonksiyonu sayesinde bu ayarlama yapılabilir.

Map fonksiyonu diğer Route fonksiyonları ile aynı yapıdadır. Constraints'ler dahil herşey aynıdır.

Aynı diğerlerinde olduğu gibi Map fonksiyonu da bir name alır.

Bunun dışında istek geldiği zaman kullanılacak bir fonksiyon ister Map fonksiyonu. Parametre olarak kullanılan bu fonksiyonun konulduğu sınıf istek neticesinde tetiklenecek sınıf olur. Bu fonksiyon delegate fonksiyonudur ileri düzey programlamada girilecek.

Bu parametre olarak verilecek olan fonksiyon lamda ile paratere olarak tanımlanabilir.

Program.cs içerisinde lambda ile tanımlanan fonksiyon, ilgili rotaya istek geldiği zaman çalışır. Fakat daha iyi bir yapı olması için bu handler fonksiyonları farklı bir sınıf içerinde de tanımlanabilir.

```
app.Map("example-route1", async c =>
{ Fonksýonun yaptığı iş});
```

Custom Rout Handler Sınıfı Oluştruma

Proje altında Handlers klasörü oluşturulur ve “isim + Handler” şeklinde isimlendirilir. Zorunlu değil. Raconsaldır. Mimari Handler sınıfını Map fonksiyonuna parametre olarak verilen fonksiyonun imzasını taşıyan sınıfı arayarak bulur.

Oluşturulan sınıfın içerisine RequestDelegate türünden Handler() isimli bir sınıf oluşturulur. Handler ismi raconsaldır.

Bu RequestDelegate sınıfı öceki partta tanımlanan fonksiyonu döndürür.

Yani işi yapacak olan fonksiyonu döndürür.

```
public class ExampleHandler
{
    0 references
    public RequestDelegate Handler()
    {
        return async c =>
        {
            await c.Response.WriteAsync("Hello from the custom route handler!");
        };
    }
}
```

Program.cs içerisinde ise parametre olarak fonksiyon verilen yere yeni bir ExampleHandler (Custom oluşturulan sınıf) sınıfı oluşturulup bu sınıf içerisinden Handler fonskiyonu çağırılır.

```
app.Map("example-route", new ExampleHandler().Handler());
```

Custom Rout Handler ile Resim Boyutlandırma Örneği

Web uygulamalarında, resimler çalışırken resimlerin HTML ile yüksekliğinin ve genişliğinin değiiştirilmesi görünüm olarak resmi küçültse de doysa boyutu olarak sabit kalmaktadır. Yani normalde 2x yer kaplayabilecek bir resim için her seferinde resmin orijinal boyutu olan 5x'i server'dan çekip HTML ile görünümünü küçültmek anlamsız bir maliyet yaratır.

İşte bu resim boyutlandırma operasyonlarını Controller'lar yerine Route Handler'lar ile yapmak daha mantıklıdır. Çünkü yaptığı tek bir iş var.

Sistem şöyle çalışacak kullanıcı(yerine göre kodu yazan kişi) "image/resim.png" url'sine gidince resmin kendisi açılacak eğer kullanıcı

"image/resim.png?w=200&h=150" gibi bir QueryString gönderirse resim istenen şekilde boyutlandırılacak.

Bu iş için bir ImageHandler sınıfı oluşturuluyor sonrasında ReguestDelegate türünden Handler isimli bir fonksiyon oluşturuluyor. Bu fonksiyon az önce de olduğu gibi delegate fonksiyonuna uygun olacak şekilde bir context dönecek task ile.

Context fonksiyonu içerisinde gerekli resim botulandırma işlemleri yapılacak.

Ama herşeyden önce Program.cs içerisinde ilgili Route Handler'I belirtmek gerekiyor.

```
app.Map("image/{imageName}", new
ImageHandler().Handler(app.Environment.WebRootPath));
```

Burada “{imageName}” önemli çünkü fonksiyon tarafında image ismini çekmek için kullanılacak. Environment.WebRootPath ise root klasörünün yolunu vermektedir fonksiyona. Fonksiyon tarafında bu iki bilgi çekilir.

ImageHandler içerisinde FileInfo sınıfı kullanılarak fonksiyona gönderilen filePath ve ImageName’I kullanarak istenilen resim bulunur.

```
FileInfo fileInfo = new
FileInfo($"{filePath}\\{c.Request.RouteValues["imageName"].ToString()}");
```

Resim bulunduktan sonra MagickImage kütüphanesini kullanarak resim için istenilen işlemler yapılabilir.

```
using MagickImage magick = new MagickImage(fileInfo);
```

Kullanıcıdan QueryString ile yükselik ve genişlik değerlerinin gelmemesine karşı resmin orjinal yükselik ve genişlik değerleri değişkenlere atanır.

```
uint width = magick.Width, height = magick.Height;
```

Gelen istek içerisinde ilgili QueryString değerleri var mı kontrol edilir. Bu kontrol için Request içerisindeki Query’nin ilgili parametreye karşılık değeri var mı kontrol edilir.

```
if (string.IsNullOrEmpty(c.Request.Query["w"].ToString()))
```

Eğer w parametresine karşılık bir veri geliyorsa

```
width = uint.Parse(c.Request.Query["w"].ToString());
```

Resmin genişlik değeri değiştirilir.

Aynı işlemler Yüseklik için de yapılır.

Değerler alındıktan sonra MagickImage kütüphanesinin Resize fonksiyonu kullanılarak resim yeniden boyutlandırılır.

```
magick.Resize(width, height);
```

bir buffer değişkeni oluşturulur ve ilgili image byte array’i olarak tanımlanır.

```
var buffer = magick.ToByteArray();
```

Ardından ne olur ne olmaz diye Response temizlenir ve Response içerisindeki ContentType'a string.Concat kullanılarak bir image yolu atanır.

```
c.Response.ContentType = string.Concat("image/", fileInfo.Extension.Replace(".",
""));
```

Son olarak await ile ilgili response ayarlamalrı yapılır ve WriteAsync kullanılarak Response yapılır.

```
await c.Response.Body.WriteAsync(buffer, 0, buffer.Length);
await c.Response.WriteAsync(filePath);
```